Update log

Update date	Update Instructions	Remarks
The 2020-08	Debut edition	
2021-04-12	1. Add time period support 2. Add the device photo inspection interface 3. Added the interface for rerecording	
2021-05-10	1, the new person-to-witness comparison mode only brushes ID card and verifies ID card 2. Add the template photo in the record upload	
2021-05-13	1. Add device prompt text	
2021-05-13	Added RTSP video stream switching function	
2021-05-24	Added features that do not require photo entry to the list	
2021-06-23	Supplement Get Video Stream Method	
2021-07-08	Added device data transmission encryption interface	
2021-08-09	Added Health Code data Push	
2021-11-24	Added parameters such as Wiegand type	
2021-12-07	Change the way the time group is set (does not affect the original setting)	
2022-03-24	Added Settings UI interface	
2023-02-20	Add face photos of capture device (Can be used to add whitelist)	
2023-03-20	Added one person multi card parameters, added some device parameters	

1 Device protocol overview

1.1 Overview

This document is an open protocol based on face recognition devices of our company. It is mainly provided to customers who need to use HTTP protocol to connect devices, and supports the development of http protocol in the local area network. The interface mainly includes personnel management, photo management, device information configuration, record uploading and other related core services.

1.2 Interface Specifications

Protocol type: The standard HTTP protocol is used. The Content type can contain Json data and binary data. For details, refer to related protocols.
Interface address: http://<ip>:8091/<URL>

Interface security: You need to use the user and password to log in to the interface for the first time. Subsequent calls to any interface need to pass the password as the interface security verification key.
Format description: Chinese characters may be involved in data communication. utf-8 format is used. When using post to transmit parameters, the parameters are put into the body in json format

1.3 Interface Description

The default boot port number is port 8091; The default password is 123456

2 Define the protocol interface

2.1 Login and heartbeat messages

2.1.1 Logging In to the Device

Url: http://deviceAddress:port/deviceLogin
Request method: post
Send the content of the message

Data type	Field name	Instructions
String	password	Default user password is 123456

Accept message response data content

Data type	Field name	Instructions
String	message	Prompt message
int	result	0 successful;

example
Url: http://192.168.1.135:8091/deviceLogin Send message: {"password":"123456"} Received successfully return message: {"message" : "Login Success","result" : 0} Receive exception return message: {"message" : "Login Failure","result" : error code}

2.1.2 Heartbeat message

Method 1: The heartBeatIp address is set by the user [Set device parameters]. The device actively uploads the heartbeat information.
Support the post method
Look at the device parameters specifically
The interface address http://deviceAddress:port/setDeviceParameter
(Interface address is the interface for configuring device parameters)
The content of the message to be sent

Data type	Field name	Instructions

String	password	Device user password
Object	data	Data that represents the current message body

Data The contents of the message body

Data type	Field name	Instructions
int	heartBeatEnable	0 On 1 off
String	heartBeatIp	Heartbeat address

Accept message answer data content

Data type	Field name	Instructions
String	message	Prompt message
int	result	0 successful; 1, the parameter setting is incorrect; 2, the password is wrong;

The heartbeat request of the device after configuration is as follows: (heartbeat interval 30S)

The content of the message sent by the device

Data type	Field name	Instructions
String	currentTime	Time
String	ip	The ip address of the device
int	personCount	Number of faces
String	SN	Device sn

Accept message answer data content

Data type	Field name	Instructions
String	message	Prompt message
int	result	0 successful; Non-0 exception

Non-0 Exception Method 2 of 2: Actively Requesting a device Heartbeat 1 Request a device heartbeat

Url: http://deviceAddress:port/heartBeat

Request method: post

Send the content of the message

Data type	Field name	Instructions
String	password	Default user password is 123456
String	ip	The ip address of the device

Accept message answer data content

Data type	Field name	Instructions
String	message	Prompt message
int	result	0 successful; 1, the parameter setting is incorrect; 2, the password is wrong;

example
Url: http://192.168.1.135:8091/heartBeat To send a message: {" password ":" 123456 ", "IP" : "192.168.1.135}" Received successfully return message: {"message" : "HeartBeat Success","result" : 0} Receive exception return message: None yet

2.2 Device Running Information

2.2.1 Obtaining Device Parameters

Url: http://deviceAddress:port/getDeviceParameter
Request method: post
Send the content of the message

Data type	Field name	Instructions
String	password	User password

Accept message response data content

Data type	Field name	Instructions
String	message	Prompt message
int	result	0 successful; 1, the parameter setting is incorrect; 2, the password is wrong;
Object	data	Data that represents the current message body

Set the contents of the data message body of the parameter:
(There is a health code setting parameter in data, don't worry about this parameter)

Data type	Field name	Instructions
String	devicename	Name
String	Mac_addr	Unique mac address of the device
String	SN	Unique serial number of device
String	location	Location
int	inout	Access 0 exit 1 entrance
String	pwd	Password 6 digits
int	whitevalue	Face recognition threshold [70,100]
int	mapvalue	Witness identification threshold [30,100]
int	recogSpaceTime	Recogspacetime Identification Removal time (seconds) [2,255]
int	delayvalue	Opening delay seconds (default 2S)

int	lightLevelPercent	Fill light light 0-100
int	lightType	0 off 1 on 2 On according to the time period 3 auto fill light (auto fill light rule: when the face is detected, the face is turned off immediately, when the camera light transmission is low often turn on the fill light)
String	lightTimeStart	Fill light turn on time
String	lightTimeEnd	Fill light off time
int	systemVol	System Volume [0,7]
int	detectRange	Recognition distance [0,3] [Unlimited,0.5 m,1 m,2 m]
int	wgOutType	Wgouttype 0. Wgouttype 26 1. Wgouttype 34
int	wgInOutMode	Wiegand Mode 0 Wiegand output, 1 Wiegand input
int	livenessEnable	Live switch 0 off 1 On
int	InfraredImaging	Black and white imaging video 0 on 1 off
int	livenessValue	Live score 0-10
int	recogModeDB	Face recognition 0 disabled, 1 enabled, 2 Health Code whitelist (This feature is face recognition when health code is not enabled)
int	recogModeIC	Recogmodeic Recognition Mode 0 Disable 1 IC card recognition 2 IC card plus face recognition
int	recogModeID	Recognition mode 0 Disable 1 ID identification (ID photo + face photo comparison) 2 ID + whitelist verification (check face in 100 list database after ID comparison) 3 Only brush ID card 4 Verify the ID card (verify the ID card number in the whitelist library after brushing the ID card)
int	detectVoiceEnable	Stranger recognition 0 disable, 1 Capture recognition, 2 Snap identification and open the door
int	recogRelay	Recognize open door 0 relay no output 1 relay output (after setting relay output not need call interface opened http://deviceAddress:port/setDeviceRemoteOpen).
String	rebootTime	Automatic restart interval "DDHHmm" DD Instructions: 00 Daily, 01 Monday, 02 Tuesday, 03 Wednesday, 04 Thursday, 05 Friday, 06 Saturday, 07 Sunday

int	TemperatureMode	Temperature measurement function 0 No Temperature measurement 1 Temperature measurement + face recognition 2 temperature measurement only
String	TempNormalMax	Temperature alarm temperature (default 37.2)
int	TempThermography	Thermographic video 0 Off 1 On
int	OutSideDetectMode	Use Environment 0: Standard Mode 1: Outdoor mode to measure temperature
int	EnableMasks	Mask detection 0 off 1 On
int	IntelligentRecognition	Hard hat recognition 0 off 1 On
int	wgOutType	Wgouttype 0- Wgouttype 26, 1-Wgouttype 34
int	wgInOutMode	Wiegand Mode 0 Wiegand output 1 Wiegand input
int	relayMode	Relay type 0 Normally on, 1 normally off
String	banner	Device banner (default value is face recognition access control terminal) (Note: To modify this value, you need to enable the bannerEnable parameter)
int	bannerEnable	Standby Screen Saver 0 off 1 On
int	bannerSpaceTime	Standby screensaver picture switching time
int	VideoSwitch	RTSP Video Streaming Toggle 0 RGB video streaming, 1 IR video
int	voiceSetting	Custom voice function 0 disabled, 1 enabled (voice module connected to serial port 2)
String	customVoice	Customize voice content
int	resultDisplayMode	Identify result display 0 disabled, 1 enabled
String	resultCustomization	Identify the results display content
int	passwordOpendoor	Passwordopendoor function 0 disabled, 1 enabled
String	doorPassword	Door opening Code (6 digits only)
Example		

Url: http://192.168.1.135:8091/getDeviceParameter
Send message: {"password":"123456"}
Received successful return message:

{
"data" :
{
"CheckTempWorkMode" : 0,
"EnableMasks" : 0,
"EnableNonResidentTemp" : 1,
"EnableTemp" : 0,
"NoMasksRing" : 1,
"NoMasksStop" : 1,
"TempBroadcast" : 1,
"TempCheckMax" : "80.00",
"TempCheckMin" : "10.00",
"TempHighRing" : 1,
"TempNormalMax" : "37.20",
"TempRingStop" : 1,
"banner" : "",
"bannerEnable" : 0,
"bannerSpaceTime" : 5,
"delayvalue" : 1,
"detectRange" : 0,
"detectVoiceEnable" : 2,
"heartBeatEnable" : 0,
"heartBeatIp" : "",
"imageQuality" : 2,
"imageSaveEnable" : 1,
"inout" : 1,
"lightLevelPercent" : 30,
"lightTimeEnd" : "23:00",
"lightTimeStart" : "19:00",
"lightType" : 0,
"livenessEnable" : 1,
"livenessValue" : 5,
"location" : "Location",
"mapvalue" : 60,
"name" : "Face recognition terminal ",
"platformEnable" : 0,
"PlatformIp" : "http://192.168.1.198:8080/getrecord/",
"pwd" : "123456",
"rebootTime" : "000000",
"recogModeDB" : 1,
"recogModeIC" : 0,

"recogModeID" : 1,
"recogRelay" : 0,
"recogSpaceTime" : 3,
"relayMode" : 0,
"resultFormat" : "",
"resultNameEncrypt" : 0,
"resultSetting" : 0,
"resultVoiceFormat" : "",
"systemVol" : 7,
"upblack" : 1,
"uploadImage" : 1,
"voiceSetting" : 2,
"wgOutType" : 0,
"whitevalue" : 80
},
"message" : "Get Device Parameter Success",
"result" : 0
}
Receive Failure return message: {"message" : "Get Device Parameter Failure","result" : error code}

2.2.2 Setting Device parameters

Url: http://deviceAddress:port/setDeviceParameter
Request method: post
Send the content of the message

Data type	Field name	Instructions
String	password	Device user password
Object	data	Data that represents the current message body

For the structure of the message content, see the content of the message body in Get Device Parameters
Accept the message response data content

Data type	Field name	Instructions
String	message	Prompt message
int	result	0 successful; 1, the parameter setting is incorrect; 2, the password is wrong;

Url: http://192.168.1.135:8091/setDeviceParameter
Send a message: {" password ":" 123456 ", "data" : {" name ":" test ", "location" : "test"}}
Receive message: {"message" : "Set Device Parameter Success","result" : 0}

2.2.3 Device Upgrade

Url: http://deviceAddress:port/uploadUpgradeAPKFile
Support post upload in form mode
Send the content of the message

Data type	Field name	Instructions
String	password	Device user password

Accept message answer data content

Data type	Field name	Instructions
String	message	Prompt message
int	result	0 successful; 1, the parameter setting is incorrect; 2, the password is wrong;

Attention:
1), Headers,Content-Type is multipart/form-data;
2), Params, need to carry password
3), Body select need to upgrade.img file, file can only be upgraded from the low version to the high version;

2.2.3 Device restart

Url: http://deviceAddress:port/setDeviceReboot
Request method: post
Send the content of the message

Data type	Field name	Instructions
String	password	Device user password
Object	data	Message body

The content of the message body

Data type	Field name	Instructions
String	type	The type is: DelayReboot Delayed reboot
int	value	Specify how many seconds after the current restart

Accept the message answer data content

Data type	Field name	Instructions
String	message	Prompt message
String	result	0 successful; 1, the parameter setting is incorrect; 2, the password is wrong;

example
Url: http://192.168.1.135:8091/setDeviceReboot Send a message: {" password ":" 123456 ", "data" : {" type ":" DelayReboot ", "value" : "5"}} Received successfully return message: {"message" : "Set Device Reboot Success","result" : 0} Error message received: {"message" : "Set Device Reboot Failure","result" : error code}

2.2.4 Device control switch on

Url: http://deviceAddress:port/setDeviceRemoteOpen

Request method: post

The content of the sent message

Data type	Field name	Instructions
String	password	Device user password

Accept message answer data content

Data type	Field name	Instructions
String	message	Prompt message
int	result	0 success; 1, the parameter setting is incorrect; 2, the password is wrong;

example

Url: http://192.168.1.135:8091/setDeviceRemoteOpen Send message: {"password":"123456"} Received successfully return message: {"message" : "Set Device Remote Open Success","result" : 0} Receive abnormal return message: {"message" : "Set Device Remote Open Failure","result" : error code}

2.2.5 Obtain version information

Url: http://deviceAddress:port/getDeviceVersion

Request method: post

Send the content of the message

Data type	Field name	Instructions
String	password	Device user password

Accept message response data content

Data type	Field name	Instructions
String	message	Prompt message
int	result	0 successful; 1, the parameter setting is incorrect; 2, the password is wrong;
Object	data	Message body

The content of the message body

Data type	Field name	Instructions
String	device_name	Device name
String	firmware_version	Software version
String	sysvertime	Compile time
String	model	Model number
String	macaddr	MAC address

Give an example

Url: http://192.168.1.135:8091/getDeviceVersion
Send message: {"password":"123456"}
Received successful return message:
{
"data": {
"device_name": "Terminal",
"firmware_version": "101.10193.0000 Build-202100607",
"macaddr": "DA:8B:11:17:23:3B",
"model": "C108LD",
"sysvertime": "20210608-093539"
},
"message": "Get Device Version Success",
"result": 0
} Receive exception return message: {"message" : "Get Device Version Failure","result" : error code}

2.2.6 Get the current picture of the camera

Url: http://deviceAddress:port/getDeviceSnapPicture
Request method: post
Send the content of the message

Data type	Field name	Instructions
String	password	Device user password

Accept message answer data content

Data type	Field name	Instructions
String	message	Prompt message
int	result	0 successful; 1, the parameter setting is incorrect; 2, the password is wrong;
Object	data	Message body

The content of the message body

Data type	Field name	Instructions
String	snapPicBase64	base64 picture data obtained by the camera

Example
Url: http://192.168.1.135:8091/getDeviceSnapPicture Send message: {"password":"123456"} Receive return message success: {" data ": {\ n \ t \" snapPicBase64 ":" / 9 j / 4 aaqskz..... vOO5waXgEEdM9R1osWhFxuXn+IAntipmh44qIADgjJ7jOasRy444pNaCP/Z"},"message" : "Get Device SnapPicture Success","result":0} Receive exception return message: {"message" : "Get Device Snap Picture Failure","result" : error code}

2.2.7 Get the current face photo of the camera

Url: http://deviceAddress:port/getDeviceSnapFace
Request method: post
This interface is used to capture the current face photo of the camera. The difference with 2.2.6 is that 2.2.6 interface directly captures the current photo after the request, 2.2.7 interface will detect the face and then capture the photo
Send the content of the message

Data type	Field name	Instructions
String	password	Device user password

Accept message answer data content

Data type	Field name	Instructions
String	message	Prompt message
int	result	0 successful; 1, the parameter setting is incorrect; 2, the password is wrong;
Object	data	Message body

The content of the message body

Data type	Field name	Instructions
String	snapPicBase64	base64 image data obtained by the camera

Example
Url: http://192.168.1.135:8091/getDeviceSnapFace Send message: {"password":"123456"} Receive return message success: {" data ": {\ n \ t \" snapPicBase64 ":" / 9 j / 4 aaqskz..... vOO5waXgEEdM9R1osWhFxuXn+IAntipmh44qIADgjJ7jOasRy444pNaCP/Z"},"message" : "Get Device SnapPicture Success","result":0} Receive exception return message: {"message" : "Get Device Snap Picture Failure","result" : error code}

2.2.7 Set the device logo picture

Url: http://deviceAddress:port/setDeviceLogo
Support the post form method to upload files
Send the content of the message

Data type	Field name	Instructions
String	password	Device user password

Accept message response data content

Data type	Field name	Instructions
String	message	Prompt message

int	result	0 successful; 1, the parameter setting is incorrect; 2, the password is wrong;

Attention:
1), Headers,Content-Type is multipart/form-data;
2), Params, need to carry password
3), Body Select the picture file you want to set

2.2.8 Restore the default device logo picture

Url: http://deviceAddress:port/restoreDeviceLogo
Request mode: post
Send the content of the message

Data type	Field name	Instructions
String	password	Device user password

Accept message answer data content

Data type	Field name	Instructions
String	message	Prompt message
int	result	0 successful; 1, the parameter setting is incorrect; 2, the password is wrong;

Url: http://192.168.2.127:8091/restoreDeviceLogo
Send data: {"password":"123456"}
Accept data: {"message":"Restore device default logo.","result":0}

2.2.9 Set the network address of the device

Url: http://deviceAddress:port/setDeviceNetwork
Request method: post
Send the content of the message

Data type	Field name	Instructions
String	password	Device user password
Object	data	Message body

Data type	Field name	Instructions
boolean	isDhcp	Whether to enable dhcp dynamic ip
String	ip	Static ip address
String	gateway	Default gateway
String	mask	Subnet mask
String	dns	dns

Accept message reply data content

Data type	Field name	Instructions

String	message	Prompt message
int	result	0 successful; 1, the parameter setting is incorrect; 2, the password is wrong;

Url: http://192.168.1.135:8091/setDeviceNetwork
To send data:
{" password ":" 123456 ", "data" : {" isDhcp ": false," IP ":" 192.168.1.136 ", "gateway" : "192.168.1.1", "mask" : "255.255.255.0", "DNS" : "8 The 8.8.8}}"
Receiving successful return message:

2.2.10 Restore all parameters of the device

Url: http://deviceAddress:port/restoreDeviceDefaultParameter
Request method: post
Note: After the parameters are restored, you need to restart the device to take effect
The contents of the message to be sent

Data type	Field name	Instructions
String	password	Device user password

Accept message answer data content

Data type	Field name	Instructions
String	message	Prompt message
int	result	0 successful; 1, the parameter setting is incorrect; 2, the password is wrong;

example

Url: http://192.168.1.135:8091/restoreDeviceDefaultParameter
Send message: {"password":"123456"}
Received successful return message:
{"message" : "Restore Device Default Parameter Success","result" : 0}
Receive an exception return message:
{"message":"Restore Device Default Parameter fail","result": error code}

2.2.12 Obtain the device time

Url: http://deviceAddress:port/getDeviceTime
Request mode: post
Send the content of the message

Data type	Field name	Instructions
String	password	Device user password

Accept message answer data content

Data type	Field name	Instructions
String	message	Prompt message

int	result	0 successful; 1, the parameter setting is incorrect; 2, the password is wrong;

example
Url: http://192.168.1.135:8091/getDeviceTime Send message: {"password":"123456"} Received successful return message: {"data" : "{\n\t\"time\" : \"2020-12-16 15:05:07\"\n}","message" : "Get Device Time Success","result" : 0} Receive exception return message: {"message" : "Get Device Time Failure","result" : error code}

2.2.13 Set device time

Url: HTTP: / / http://deviceAddress:port/setDeviceTime
Request method: post
Send the content of the message

Data type	Field name	Instructions
String	password	Device user password

Accept message answer data content

Data type	Field name	Instructions
String	message	Prompt message
int	result	0 successful; 1, the parameter setting is incorrect; 2, the password is wrong;

example
Url: http://192.168.1.135:8091/setDeviceTime Sends a message: {" password ":" 123456 ", "data" : {" time ":" the 2020-06-12 18:00:00 "}} Received successfully return message: {"message" : "Set Device Time Success","result" : 0} Error message {"message" : "Set Device Reboot Failure","result" : error code}

2.2.14 Set device firmware upgrade

Description: Setting device firmware upgrade has the same function as device upgrade
Url: http://deviceAddress:port/setDeviceOTAUpgrade
Support post upload expression
Send the content of the message

Data type	Field name	Instructions
String	password	Device user password

Accept message answer data content

Data type	Field name	Instructions
String	message	Prompt message

int	result	0 successful; 1, the parameter setting is incorrect; 2, the password is wrong;

Attention:
1), Headers,Content-Type is multipart/form-data;
2), Params, need to carry password
3), Body selects update.zip file that needs to be upgraded from a low version to a high version;

2.2.15 The device displays prompt text

Url: http://deviceAddress:port/setDeviceShowMessage
Request method: post
When this interface is called, the device will pop up to display the content delivered by the interface for three seconds.
The content of the message to be sent

Data type	Field name	Instructions
String	password	Default user password is 123456
Object	data	Data that represents the current message body

The contents of the data message body of the argument:

Data type	Field name	Instructions
String	message	Send a maximum of 14 kanji characters

Accept message response data content

Data type	Field name	Instructions
String	message	Prompt message
int	result	0 successful;

example

Url: http://192.168.1.135:8091/setDeviceShowMessage
Sends a message: {" password ":" 123456 ", "data" : {" message ":" this is the test content "}}
Received successfully return message: {"message" : "Set Device Show Message Success","result" : 0}
Receive exception return message: {"message" : "Login Failure","result" : error code}

2.2.16 Obtain the current card number of the device

Url: http://deviceAddress:port/getDeviceIcidNumber
Request mode: post
Send the content of the message

Data type	Field name	Instructions
String	password	Device user password

Accept message answer data content

Data type	Field name	Instructions
String	message	Prompt message
int	result	0 successful; 1, the parameter setting is incorrect; 2, the password is wrong;
Object	data	Message body

The content of the message body

Data type	Field name	Instructions
String	icidNumber	The IC card number /ID card number/Wiegand number read by the device

Example
Url: http://192.168.1.135:8091/getDeviceSnapPicture Send message: {"password":"123456"} Received successfully return message: {"},"message" : "Get Device SnapPicture Success","result":0} Receive exception return message: {"message" : "Get Device Snap Picture Failure","result" : error code}

2.2.17 Set device UI

Url: http://deviceAddress:port/setDeviceUI

Support the post form method to upload files

Send the content of the message

Data type	Field name	Instructions
String	password	Device user password

Accept message answer data content

Data type	Field name	Instructions
String	message	Prompt message
int	result	0 successful; 1, the parameter setting is incorrect; 2, the password is wrong;

Attention:
1), Headers,Content-Type is multipart/form-data; 2), Params, need to carry password 3), Body Select the picture file you want to set Note that the setup UI must pass a PNG image

2.3 List Management Interface

2.3.1 Adding a List

Url: http://deviceAddress:port/addDeviceWhiteList

Request method: post

Send the content of the message

Data type	Field name	Instructions
int	totalnum	Totalnum Indicates the total number of whitelists that need to be synchronized
int	currentnum	Indicates the number of the current synchronization whitelist, starting with 1
Object	data	Message body

The content of the message body

Data type	Field name	Instructions
String	usertype	Y user type white(whitelist), black (black list), visitor (visitor)
String	employee_number	Y Person number
String	name	N Name
String	playname	N Broadcast name (applicable to universal voice polyphonics, universal voice non-standard, please ask business personnel before docking, non-polyphonics do not issue this field)
String	sex	N Male, female
String	nation	N Nation
String	telephone	N Telephone
String	idno	N ID number
String	idstartdate	N ID validity period begins
String	idenddate	N End of validity of ID card
String	peoplestartdate	N Time when the list validity period starts
String	peopleenddate	N End date of the list validity period
String	idissue	N ID card issuing authority
String	idaddress	N ID address
String	icno	N ic card number (10-digit decimal)
String	icno2	N ic card number
String	icno3	N ic card number
String	icno4	N ic card number
String	QRcode	N QR code

String	company	N Company
String	department	N department
boolean	passAlgo	Y true: No photo required false: A photo is required
int	TimeGroupId	Ordinary time period 0: Any time can successfully identify the peer, 1-4 Monday to Sunday within the specified time period to successfully identify the pass
int	SpecialGroupId	Special Time Period ID (Traffic can be identified during this time period)
String	register_base64	The base64 string representing the list picture

Accept message response data content

Data type	Field name	Instructions
String	message	Prompt message
int	result	0 successful; 1, the parameter setting is incorrect; 2, the password is wrong;

Url: http://192.168.1.135:8091/addDeviceWhiteList
To send a message:
{"password":"123456",
"totalnum": 1,
"currentnum": 1,
"data": {
"usertype":"white",
"idno":"431024199522140067",
"employee_number": "1001",
"name": "Li Si ",
"sex": "female ",
"nation": "",
"idstartdate": "",
"idenddate": "",
"peoplestadate":"2020-12-16",
"peopleenddate":"2023-12-16",
"idissue":"",
"idaddress":"",
"icno": "",
"QRcode": "",
"passAlgo": false,
"company": "",
"department": "",
"TimeGroupId": 0,
"register_base64": "/9j……."
}
}
Received successfully return message: {"message" : "Add successfully ","result" : 0}
Receive exception return message: {"message" : "error message","result" : 1}
Note:
When delivering the list, in order to deliver the list progress bar to the interface, it is recommended to fill totalnum and currentnum as required;

2.3.2 Deleting the list

Url: http://deviceAddress:port/deleteDeviceWhiteList
Request method: post
Send the content of the message

Data type	Field name	Instructions
String	password	Device user password
Object	data	Message body

The content of the message body

Data type	Field name	Instructions
String	employee_number	Y Person number
String	usertype	Y user type white(whitelist),black (blacklist)

Accept message response data content

Data type	Field name	Instructions
String	message	Prompt message
int	result	0 success; 1, the parameter setting is incorrect; 2, the password is wrong;

Url: http://192.168.1.135:8091/deleteDeviceWhiteList
Send a message: {" password ":" 123456 ", "data" : {" employee_number ":" 1001 ", "usertype" : "white"}}
Received successfully return message: {"message" : "Delete White List Success","result" : 0}
Receive exception return message: {"message" : "Delete White List Failure","result" : error code}

2.3.3 Delete all whitelists

Url: http://deviceAddress:port/deleteDeviceAllWhiteList
Request method: post
Send the content of the message

Data type	Field name	Instructions
String	password	Device user password

Accept message answer data content

Data type	Field name	Instructions
String	message	Prompt message
int	result	0 successful; 1, the parameter setting is incorrect; 2, the password is wrong;

Url: http://192.168.1.135:8091/deleteDeviceAllWhiteList
Send message: {"password":"123456"}
Receive message: {"message" : "Delete All Device White List Success","result" : 0}

2.3.4 Get the list list

Url: http://deviceAddress:port/getAllDeviceIdWhiteList
Request method: post
Send the content of the message

Data type	Field name	Instructions
String	password	Device user password

Accept message answer data content

Data type	Field name	Instructions
String	message	Prompt message
int	result	0 successful; 1, the parameter setting is incorrect; 2, the password is wrong;
Object	data	Message body

The content of the message body

Data type	Field name	Instructions
String[]	idNumList	A uniquely numbered array representing people

Example

Url: http://192.168.1.135:8091/getAllDeviceIdWhiteList
Send message: {"password":"123456"}
Received successful return message:
{
"data":"{\n\t\"idList\":\n\t[\n\t\t\"3886\",\n\t\t\"3887\",\n\t\t\"3888\"}",
"message" : "Get All Device White List Success",
"result" : 0
}
Receive exception return message: {"message" : "Get All Device White List Failure","result" : error code}

2.3.5 Get list details

Url: http://deviceAddress:port/getDeviceWhiteListDetailByIdNum

Request method: post

Send the content of the message

Data type	Field name	Instructions
String	password	Device user password
Object	data	Message body

The content of the message body

Data type	Field name	Instructions
String	employee_number	Represents the unique number of the person

Accept message response data content

Data type	Field name	Instructions
String	message	Prompt message
int	result	0 successful; 1, the parameter setting is incorrect; 2, the password is wrong;
Object	data	Message body

The content of the message body

Data type	Field name	Instructions
String	usertype	user type; white(whitelist),black (blacklist)
String	name	Name
String	sex	Male, female
String	nation	Nation
String	idno	Id number
String	peoplestartdate	List expiration date Begins
String	peopleenddate	End of list expiration date
String	idissue	Identity card issuing authority
String	idaddress	Id address
String	icno	ic card number 20180716
int	TimeGroupId	0: Any time identification success can be peer 1: Monday to Sunday in the specified period of time to successfully identify to pass, outside the period prompt "no permission to enter, please identify in the specified period"
String	register_base64	base64 string of the list

Url: http://192.168.1.135:8091/getDeviceWhiteListDetailByIdNum
Send a message: {" password ":" 123456 ", "data" : {" employee_number ":" 5463 "}}
Receiving successful return message: {

"data" :
{
"TimeGroupId" : 0,
"icno" : "0020202469",
"idaddress" : "",
"idissue" : "",
"idno" : "",
"name" : "2573",
"nation" : "",
"peopleenddate" : "2030-12-18 15:18:20",
"peoplestartdate" : "2020-12-18 15:18:20",
"register_base64" : "/9j/4AAQSkZ...." ,
"sex" : "male ",
"usertype" : "white"
},
"message" : "Get Device White Detail By ID Number Success",
"result" : 0

}
Receive exception put back message: {"data" : null,"message" : "Get Device White Detail By ID Number Failure","result" : 1}

2.3.6 Verifying image quality

Url: http://deviceAddress:port/checkFacePicture
Request method: post
Send the content of the message

Data type	Field name	Instructions
String	password	Device user password
Object	data	Message body

The content of the message body

Data type	Field name	Instructions
String	register_base64	base64 string of the list

Accept the message answer data content

Data type	Field name	Instructions
String	message	Prompt message
int	result	0 Verification succeeds. 1 photo verification failed;

2.4 Standby picture setup interface

2.4.1 Delete all standby pictures

Url: http://deviceAddress:port/deleteAllDeviceAdvertPicture
Request method: post
Send the content of the message

Data type	Field name	Instructions
String	password	Device user password

Accept message answer data content

Data type	Field name	Instructions
String	message	Prompt message
int	result	0 successful; 1, the parameter setting is incorrect; 2, the password is wrong;

Note: all images of the device will be cleared at once

Url: http://192.168.2.127:8091/deleteAllDeviceAdvertPicture
Send message: {"password":"123456"}
Receive message: {"message":"Delete all advert pictures success","result":0}

2.4.2 Add a standby picture

Url: http://deviceAddress:port/addDeviceAdvertPicture
Request method: post
Send the content of the message

Data type	Field name	Instructions
String	password	Device user password

Accept message answer data content

Data type	Field name	Instructions
String	message	Prompt message
int	result	0 successful; 1, the parameter setting is incorrect; 2, the password is wrong;

Attention:
1), Headers,Content-Type is multipart/form-data;
2), Params, need to carry password
3), limit the size of uploaded picture files to 400K or less;

2.5 log callback interface

2.5.1 Obtaining device logs

Url: http://deviceAddress:port/getDeviceLog
Request method: post
Send the content of the message

Data type	Field name	Instructions
String	password	Device user password
Object	data	Message body

The content of the message body

Data type	Field name	Instructions
String	devType	Describe the type of device diary, handle means obtaining the operation diary;

Accepts the message reply data content

Data type	Field name	Instructions
String	message	Prompt message
int	result	0 successful; 1, the parameter setting is incorrect; 2, the password is wrong;

When it is wrong, an error message will be returned accordingly; When successful, the diary file will be directly transmitted by data;

Can be based on the actual effect

Url: http://192.168.1.135:8091/getDeviceLog
To send a message:
{"password":"123456", "data":{"devType":"handle"}}
{"password":"123456","data":{"devType":"crash"}}
Receive message: The actual data may prevail

2.5.2 Deleting Device Logs

Url: http://deviceAddress:port/deleteDeviceLog

Request method: post

Send the content of the message

Data type	Field name	Instructions
String	password	Device user password
Object	data	Message body

The content of the message body

Data type	Field name	Instructions
String	devType	Describe the type of device diary, handle means obtaining the operation diary;

Accepts the message reply data content

Data type	Field name	Instructions
String	message	Prompt message

int	result	0 successful; 1, the parameter setting is incorrect; 2, the password is wrong;

Url: http://192.168.1.135:8091/deleteDeviceLog
Send a message: {" password ":" 123456 ", "data" : {" devType ":" crash "}}
Received successfully return message: {"message" : "Delete Device Log Success","result":0}
Receive abnormal return message: {"message" : "Delete Device Log Failure","result" : error code}

2.6 Get identification records

2.6.1 Callback interface for identifying results

Url: http://deviceAddress:port/setIdentifyCallBck
Request mode: post
When platformEnable=1 and platformIp information are configured, the recognition record will be actively sent to the address after face recognition
(The record supports disconnection and continuation)

Data type	Field name	Instructions
String	password	123456
int	platformEnable	Upload identification record to platform 0 off 1 on
String	platformIp	Platform Url address

Normal return: {"message":"Set access groups success","result":0}
{"message":"Set access groups fail","result": error code}
The push format is as follows: (must set up http server to receive data, and make sure to disable firewall, antivirus software, driver manager software)

Data type	Field name	Required	Instructions
String	id	Y	uuid
String	Mac_addr	Y	Device unique identification code
String	time	Y	Compare time yyyy-MM-dd HH:mm:ss
String	devicename	N	Device name
String	location	N	Installation location
int	inout	N	Access 0 Exit 1 entrance
String	employee_number	Y	Personnel number
String	name	N	Name
String	sex	N	Gender

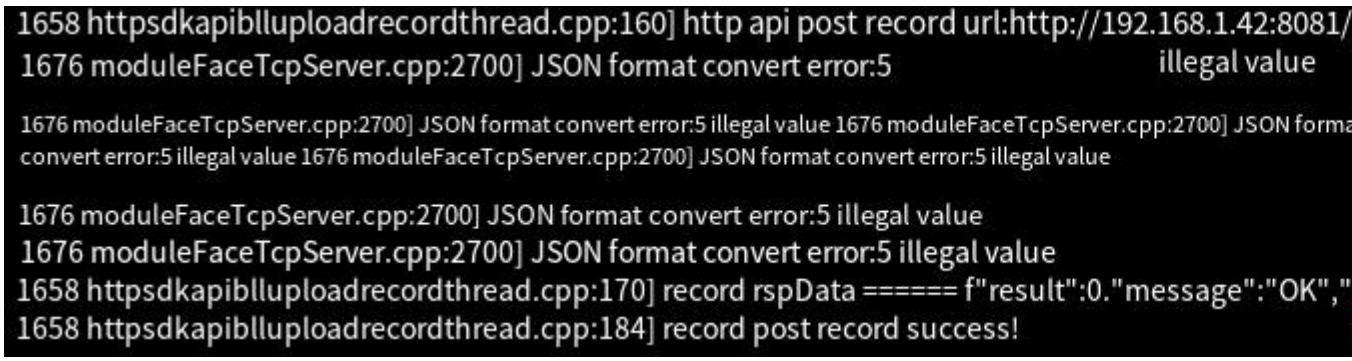
String	nation	N	Nationality
String	idNum	N	Id number
String	icNum	N	IC card number
String	birthday	N	Date of birth: April 3, 1997
String	telephone	N	Phone
String	address	N	Address information on your ID card
String	depart	N	Issuing authority
String	validStart	N	Start of expiration date
String	validEnd	N	Expiration date
int	IdentifyType	Y	Identification method (comparison type) : 0 face recognition, 1 blacklist recognition (reserved field), 2 witness comparison, 3 IC card recognition
int	resultStatus	Y	Comparison result 1 Successful comparison 0 failed comparison
String	face_base64	Y	Compare the snap photo base64 bit string
String	templatePhoto	N	Stencil photo
String	temperature	N	Body temperature
int	healthCode	N	Health Code Type 0 Disabled 1 Green code 2 Yellow code 3 Red code
json	rna	N	Other test information
Int	rna.ret	N	Nucleic acid test results (0 not turned on, 1 negative, 2 positive)
String	rna.time	N	Nucleic acid test time
String	antiboby.ret	N	Vaccinate or not 0 is not vaccinated, 1 is vaccinated
String	antiboby.time	N	Inoculation time yyyy-mm-dd hh:mm:ss
String	antiboby.src	N	Vaccination records

Url: http://192.168.1.135:8091/setIdentifyCallBck
To send a message:
{" password ":" 123456 ", "platformEnable" : 1, "platformIp" : "http://192.168.1.105:8080/getrecord/"}
Received successfully return message: {"message" : "set identify callback success","result" : 0}
Error message received: {"message":"Set access groups fail","result": error code}

After the active push upload is recorded, the server should reply after receiving each piece of data

```
{
 "result":0, //0 Received successfully, non-0 received failed, the device will
push again
  "message": "OK"
}
```

If you do not reply, the device will continue to push the same record.
The following logs are printed by the device. You can obtain the logs to view during debugging



2.6.1 Extracting the identification result again

Extract all the records in a certain period of time. After receiving this command, the device will upload all the records in this period to the platformIp address one by one.
Url: http://deviceAddress:port/setDeviceRecordRevert
Request method: post
Content-Type:application/json
Send the content of the message

Data type	Field name	Instructions
String	password	Device user password
Object	data	Message body

The content of the message body

Data type	Field name	Instructions
long	startTime	The start time of the record
long	endTime	The end time of the record

Accept the message answer data content

Data type	Field name	Instructions
String	message	Prompt message
int	result	0 successful; 1, the parameter setting is incorrect; 2, the password is wrong;

Url: http://192.168.2.127:8091/setDeviceRecordRevert
To send a message:
{"password":"123456", "data":{"startTime":"2020-10-01 00:00:00","endTime":"2021-04-05 23:59:59"}}
Accept message: {"message":"Success","result":0}

2.7 Get the device real-time video stream

The default video stream output of the system is off. Please modify the video stream output through the following interfaces
If the video output function is enabled on the T92K and T91K devices, the scheduled restart must be enabled every day. Otherwise, the devices may restart
Url: http://deviceAddress:port/setDeviceParameter
Request method: post
Send the content of the message

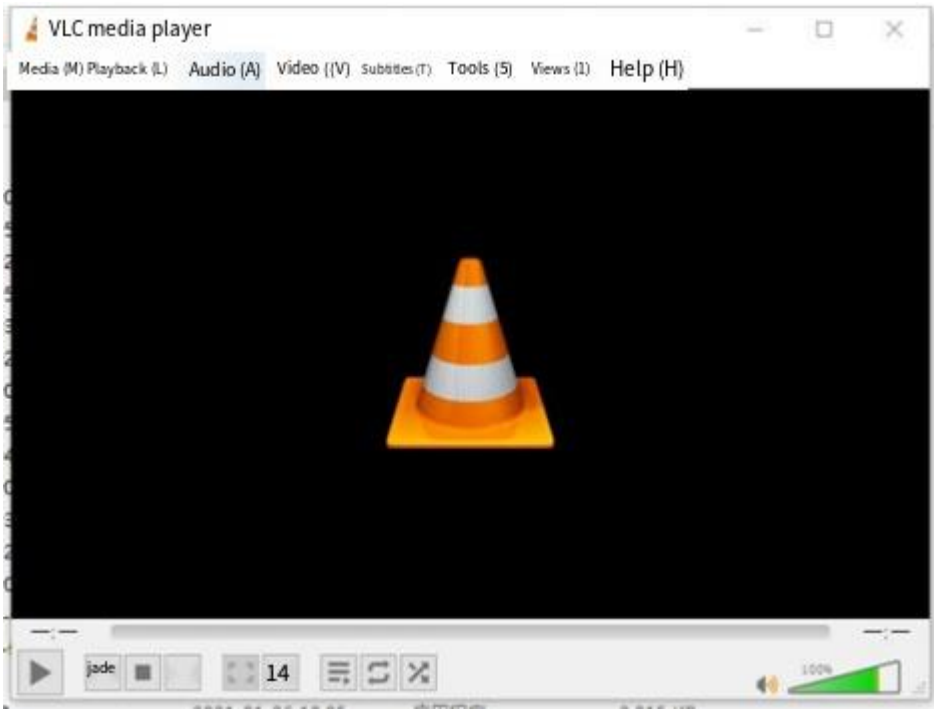
Data type	Field name	Instructions
String	password	Device user password
Object	data	Data that represents the current message body
int	VideoSwitch	Video Streaming Switch 0 Turn on video stream 2 Turn off video stream

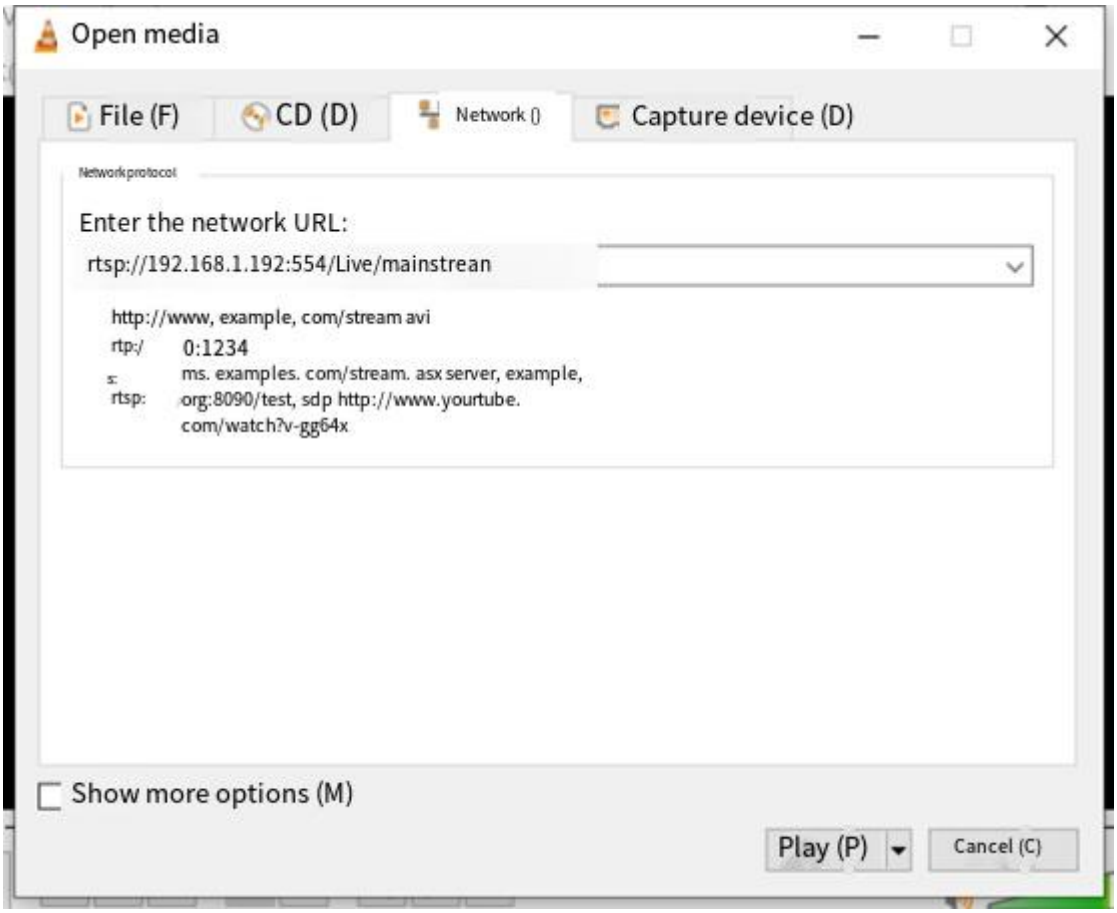
For the structure of the message content, see the content of the message body in Get Device Parameters
Accept the message response data content

Data type	Field name	Instructions
String	message	Prompt message
int	result	0 successful; 1, the parameter setting is incorrect; 2, the password is wrong;

Url: http://192.168.1.135:8091/setDeviceParameter
Send a message: {" password ":" 123456 ", "data" : {" VideoSwitch ": 0}}
Receive message: {"message" : "Set Device Parameter Success","result" : 0}

Device support to get RTSP video stream, test get method
rtsp://ip:554/live/mainstream





Test method:

1. Install the VLC tool
2. Open VLC
3. Media - Open Web streaming

Input RTSP: / / IP: 554 / live/mainstream

Just play it.

2.8 Other Messages

2.8.1 Setting the Access Control Period

Function Description: It can be from Monday to Sunday, and a maximum of four time periods are set as the allowed time period every day. Each time group has a unique ID, and the personnel associated with this ID are restricted by this time group.
Url: http://deviceAddress:port/setDeviceTimeAccessGroups
Request method: post
Content-Type:application/json
Send the content of the message

Data type	Field name	Instructions
String	password	Device user password
Object	data	Message body

data The contents of the message body

Data type	Field name	Instructions
List<WeekTime>	TimeGroup	

Definition of the WeekTime structure

Data types	Field name	Instructions
String	TimeGroupId	Time group unique ID
List<DayTimes>	TimeGroup	

Definition of DayTimes structure

Data types	Field name	Instructions
String	WeekIndex	Corresponding weeks: 0 Monday, 1 Tuesday, 2 Wednesday, 3 Thursday, 4 Friday, 5 Saturday, 6 Sunday
List<TimesBean>	Times	

TimesBean structure definition

Data types	Field name	Instructions
String	Start	Start time format Example: 09:00
String	End	End format Example: 18:00

Accept message answer data content

Data type	Field name	Instructions

String	message	Prompt message
int	result	0 successful; 1, the parameter setting is incorrect; 2, the password is wrong;

ACS AUTO

Url: http://192.168.2.127:8091/setDeviceTimeAccessGroups
To send a message:


```
{
"password": "123456",
"data": {
"TimeGroup": [{
"TimeGroupId": "1",
"TimeGroup": [{
"WeekIndex": "0",
"Times": [{
"Start": "08:00",
"End": "12:00"
}, {
"Start": "14:00",
"End": "18:00"
}, {
"Start": "19:00",
"End": "22:00"
}]
}, {
"WeekIndex": "1",
"Times": [{
"Start": "08:00",
"End": "12:00"
}, {
"Start": "14:00",
"End": "18:00"
}, {
"Start": "19:00",
"End": "22:00"
}]
}, {
"WeekIndex": "2",
"Times": [{
"Start": "08:00",
"End": "12:00"
}, {
"Start": "14:00",
"End": "18:00"
}, {
"Start": "19:00",
"End": "22:00"
}]
}, {
```

"WeekIndex": "3",
"Times": [{
"Start": "08:00",
"End": "12:00"
}, {
"Start": "14:00",
"End": "18:00"
}, {
"Start": "19:00",
"End": "22:00"
}]
}, {
"WeekIndex": "4",
"Times": [{
"Start": "08:00",
"End": "12:00"
}, {
"Start": "14:00",
"End": "18:00"
}, {
"Start": "19:00",
"End": "22:00"
}]
}, {
"WeekIndex": "5",
"Times": [{
"Start": "08:00",
"End": "23:00"
}]
}, {
"WeekIndex": "6",
"Times": [{
"Start": "08:00",
"End": "23:00"
}]
}]
}]
}
}
Accept message: {"message":"Success","result":0}

2.8.1 Set a special time period

You can set more than one special time period to set a specific date and time, whether to allow or deny access

Note: The system gives priority to judge the special time and control the passage

Url: http://deviceAddress:port/setDeviceAccessSpecialTimeGroup

Request method: post

Content-Type:application/json

Send the content of the message

Data type	Field name	Instructions
String	password	Device user password
Object	data	Message body

data The contents of the message body

Data type	Field name	Instructions
List<SpecialTimeGroupData>	SpecialTimeGroup	

Definition of the SpecialTimeGroupData structure

Data type	Field name	Instructions
String	SpecialTimeGroupId	Special Time Period ID (Less than or equal to 0, indicating no restricted access)
List<TimesBean>	SpecialTimeGroupData	

Definition of TimesBean structure

Data types	Field name	Instructions
long	Start	Represents the number of milliseconds of the start time
long	End	The number of milliseconds that represent the end time
int	PassStatus	0 Passstatus, 1 prohibit

Accept message reply data content

Data type	Field name	Instructions	
String	message	Prompt message	
int	result	0 successful; 1, the parameter setting is incorrect; 2, the password is wrong; 3, the user failed to set the password	

```
{
"password": "123456",
"data": {
"SpecialTimeGroup": [{
"SpecialTimeGroupId": "1",
"SpecialTimeGroupData": [{
"Start": "1577694600000",
"End": "1577700000000",
"PassStatus": 0
}]
}, {
"SpecialTimeGroupId": "11",
"SpecialTimeGroupData": [{
"Start": "1573207440000",
"End": "1573207920000",
"PassStatus": 1
}, {
"Start": "1573270380000",
"End": "1573270380000",
"PassStatus": 1
}, {
"Start": "1573439520000",
"End": "1573439520000",
"PassStatus": 0
}]
}]
}
}
```


2.9 Encryption of device data transmission

The encryption of this interface is to make the system and equipment exclusive sales, to avoid the market with the same manufacturer of equipment cross-goods.
After this interface is applied, the upload record of the cloud protocol will also be encrypted and transmitted.

2.9.1 Write the device verification code

The writing of the check code is irreversible. Do not attempt it easily
Url: http://deviceAddress:port/ SetCheckCode
Request method: post
The content of the sent message

Data type	Field name	Required	Instructions	Precautions
String	Crc	Y	Check code	

String	key	N	Secret key	After the key and iv fields are written, the device will use aes cbc mode 128-bit encryption to record the personnel number and comparison time of the upload interface
String	iv	N	Offset (less than 16 bytes in length)	
String	password	Y	The default password is 123456	

Accept message response data content

Data type	Field name	Instructions
String	message	Prompt message
int	result	0 successful;

2.9.2 Obtain the device verification code

Url: http://deviceAddress:port/ GetCheckCode
Request method: post
Send the content of the message

Data type	Field name	Instructions
String	password	Default user password is 123456

Accept message response data content

Data type	Field name	Instructions
String	message	Prompt message
int	result	0 successful; 1, the parameter setting is incorrect; 2, the password is wrong;
Object	data	Message body

data The contents of the message body

Data type	Field name	Instructions
String	SN	Unique serial number of device
String	Crc	Check code

3.0. Online comparison identification

Purpose: Push the data to the platform address after device identification, and return the information to the device for door opening and prompt after recognition and judgment

The online interface can be run independently from the whole LAN, and the configuration tool can directly switch the comparison mode and configure the online comparison address.
This interface can be understood as a separate set of protocols
Request URI

- /api/OnlineComparison/

Request method
http post
Device Request parameters:

Parameters	Parameter types	Must or not	Paraphrase
deviceId	string	is	Device Unique Identification (mac)
dev_sno	string	is	Device number (Device MAC address)
deviceSN	string	is	Device SN
token	string	no	token
type	string	is	How to identify: type= stranger, which stands for stranger type=face, face recognition; type=qr, representing scan code recognition; type=id, which represents the comparison between witnesses and witnesses; type=ic, representing swipe card comparison;
value	string	is	Identify the content: When type= stranger, the value content is the fixed character "data.capture" When type=face, the value content is the person id number; When type=qr, the value content is the content string of the recognized two-dimensional code; When type=id, the value content is the ID card number recognized by the ID card;
temperature	string	no	Recognize temperature values and use them as background records only;
data	object	no	Data
data.capture	string		Live snap photo, base64 encoded
data.time	string		Identify the time, with a timestamp accurate to milliseconds

Sample request:

```
{
        "dev_sno": "04:0C:F4:12:68:05",
        "deviceId": "0A:0C:F4:12:68:05",
        "devicesN": "T02F11601FS",
        "temperature": "",
        "token": "dc70ace460a00a093b51efff3588d47b",
        "type": "",
        "value": "",
        "data": {
                "capture": "",
                "datetime": ""
        }
}
```

Platform return argument:

Parameters	Parameter types	Paraphrase
status	int	0: successful; 1: failure (non-zero failure). Open the door on success, not on failure.
msg	String	Prompt message. If the content is returned, it is displayed, if it is not returned, it is not displayed.
audio	String	Voice (need to use smart voice module, voice module connected to serial port 2, this field is not applicable to T12)

```
{
        "status": 0,
        "msg": "Recognition success",
    "audio": "Identification success"
}
```

Cloud Protocol

If the LAN protocol does not apply, check the cloud protocol
https://shimo.im/docs/dPkpKzQMb7iybNqO